

Sistema de Gestão de Caixas d'Água

Kotlin + PostgreSQL — Guia completo de estudo

Arquitetura, código comentado, conceitos e 40 perguntas com resposta

1. Resumo em 30 segundos

O sistema representa uma **empresa que vende e instala caixas d'água**. Ele roda no console, guarda tudo no PostgreSQL e controla quatro coisas:

- Pessoas** — funcionários (divididos em 4 setores), clientes e fornecedores.
- Produto** — a caixa d'água, com controle de estoque.
- Serviço** — a instalação da caixa na casa do cliente.
- Dinheiro** — todo real que entra ou sai passa por um único objeto, o `Caixa`.

A frase que resume o trabalho: "Nenhuma parte do sistema mexe em dinheiro sozinha. Toda operação financeira passa pelo `object Caixa`, que valida, aplica o sinal, grava no banco e só então atualiza o saldo."

2. Arquitetura em camadas

O projeto está separado em camadas para que cada arquivo tenha **uma responsabilidade só**. Isso é o que se chama de separação de responsabilidades.

Camada	Pacote	O que faz
Apresentação	<code>sistema/</code>	Mostra os menus e conversa com o usuário. Não sabe SQL e não calcula saldo.
Validação	<code>validacao/</code>	O "porteiro". Todo dado digitado passa por aqui antes de entrar no sistema.
Domínio / Modelo	<code>pessoas/</code> , <code>produto/</code> , <code>financeiro/</code>	As classes que representam o negócio e guardam as regras dele.
Enumeradores	<code>enumeradores/</code>	Valores fixos e fechados (cores, materiais, setores, tipos de movimentação).
Repositório / DAO	<code>repositorio/</code>	Único lugar que escreve SQL. Traduz objeto Kotlin $\rightleftharpoons$ linha do banco.

Se perguntarem "por que separar em camadas?" — Porque se um dia a empresa trocar o PostgreSQL por MySQL, só o pacote `repositorio` muda. Os menus e as classes de negócio continuam iguais. E se a interface virar web, só o pacote `sistema` muda.

Mapa dos arquivos

```
src/
├─ Main.kt                ponto de entrada – try/catch/finally global
├─ enumeradores/
│   ├── Cor.kt · Material.kt · Turno.kt · Habilidade.kt
│   ├── Setor.kt          4 setores + quem pode aprovar caixa
│   └─ TipoMovimentacao.kt 6 tipos + sinal (+1 entra / -1 sai)
├─ pessoas/
│   ├── Pessoa.kt         classe-MÃE (open)
│   ├── Funcionario.kt    : Pessoa
│   ├── Instalador.kt     : Funcionario (3º nível de herança)
│   ├── Cliente.kt        : Pessoa
│   └─ Fornecedor.kt     : Pessoa
├─ produto/
│   ├── CaixaDaAgua.kt    estoque com escrita privada
│   └─ Servico.kt         status controlado
├─ financeiro/
│   ├── Movimentacao.kt   os 6 dados exigidos, imutáveis
│   └─ Caixa.kt           SINGLETON – saldo privado
├─ repositorio/
│   ├── ConexaoPostgres.kt abstract
│   ├── InterfaceJPA.kt   interface genérica <T>
│   └─ CRUD*.kt           6 repositórios
├─ validacao/Validador.kt REGEX + TRY + NULLABLE
└─ sistema/
    ├── Sessao.kt         quem opera = o responsável
    └─ Menu*.kt           5 telas de menu
banco/schema.sql         6 tabelas + view + dados de teste
```

3. Mapa dos requisitos do enunciado

Requisito	Onde está	Como foi feito
Menus interativos via console	<code>sistema/Menu*.kt</code>	Menu principal + 4 submenus, todos em <code>do/while</code> com <code>when</code> .
Todos os dados persistidos no banco	<code>repositorio/</code>	6 classes CRUD com JDBC puro e <code>PreparedStatement</code> .
Fluxo de produto (compra, venda, estoque)	<code>MenuFinanceiro.kt</code>	Comprar sobe estoque e tira dinheiro; vender desce estoque e põe dinheiro.
Fluxo de serviço (montagem/instalação)	<code>MenuServico.kt</code>	Agendar → concluir (gera receita) → ou cancelar.
Gerenciar as pessoas do negócio	<code>pessoas/</code> + <code>MenuPessoas.kt</code>	CRUD completo de funcionário, cliente e fornecedor.
Funcionários em setores (mínimo 2)	<code>enumeradores/Setor.kt</code>	4 setores: Financeiro, Administrativo, Logística e Instalação.
Fluxo de caixa seguro (encapsulamento)	<code>financeiro/Caixa.kt</code>	<code>object</code> singleton com <code>private var saldoInterno</code> .
Quanto dinheiro foi usado	<code>Movimentacao.valor</code>	<code>BigDecimal</code> , coluna <code>NUMERIC(12,2)</code> .
Quem foi o pagador	<code>Movimentacao.pagador</code>	<code>VARCHAR(120) NOT NULL</code> .
Quem recebeu	<code>Movimentacao.recebedor</code>	<code>VARCHAR(120) NOT NULL</code> .
A data e a hora	<code>Movimentacao.dataHora</code>	<code>LocalDateTime</code> ⇔ <code>TIMESTAMP</code> .
O motivo / descrição	<code>Movimentacao.motivo</code>	<code>VARCHAR(200) NOT NULL</code> .
Um responsável pela transação	<code>Movimentacao.responsavel</code>	Vem do <code>object Sessao</code> — o funcionário logado.
À prova de falha humana (REGEX/TRY/NULLABLE)	<code>Validador.kt</code> + <code>schema.sql</code>	Validação dupla: no Kotlin e nos <code>CHECK</code> do banco.

4. O banco de dados

Tabela	Guarda	Detalhe importante
fornecedor	Empresas de quem compramos	CNPJ <code>UNIQUE</code> + <code>CHECK</code> regex.
funcionario	Quem trabalha na empresa	Coluna <code>setor</code> com <code>CHECK IN</code> dos 4 setores.
cliente	Quem compra de nós	Coluna <code>parcelas_a_pagar</code> <code>NUMERIC(12,2)[]</code> — um array .
caixa_da_agua	O produto	<code>dimensao</code> <code>FLOAT8[]</code> com <code>CHECK</code> de exatamente 3 medidas.
movimentacao	Todo o dinheiro	As 6 colunas exigidas. Só recebe INSERT, nunca UPDATE/DELETE.
servico	As instalações	3 chaves estrangeiras: cliente, instalador e caixa.
v_saldo_caixa	(VIEW) saldo consolidado	Consulta salva que soma entradas e saídas.

Por que os tipos escolhidos

Tipo SQL	Tipo Kotlin	Por quê
<code>SERIAL PRIMARY KEY</code>	<code>Int</code>	O banco gera o ID sozinho, sem risco de duplicar.
<code>NUMERIC(12,2)</code>	<code>BigDecimal</code>	Dinheiro nunca usa Float/Double. Double erra centavos: <code>0.1 + 0.2 = 0.30000000000000004</code> .
<code>TIMESTAMP</code>	<code>LocalDateTime</code>	Guarda data e hora, como o enunciado exige.
<code>DATE</code>	<code>LocalDate</code>	A data da instalação não precisa de hora.
<code>FLOAT8[]</code>	<code>MutableList<Double></code>	Guarda as 3 dimensões numa coluna só.
<code>VARCHAR + CHECK IN</code>	<code>enum class</code>	Espelha o enum, impedindo valor inventado no banco.

Pergunta quase certa: "Por que `BigDecimal` e não `Double` ?"

Porque `Double` é binário e não representa exatamente os décimos e centésimos do sistema decimal. Em contabilidade, um centavo de erro repetido mil vezes vira um prejuízo real. `BigDecimal` guarda o número exato e deixa você escolher o arredondamento (`setScale(2, HALF_UP)`).

5. Conceitos de POO no código

5.1 Herança

Pessoa → Funcionario → Instalador

```
// Pessoa.kt – "open" é OBRIGATÓRIO em Kotlin para poder herdar.  
// Sem "open", toda classe Kotlin é final (fechada).  
open class Pessoa(  
    val nome: String,  
    val documento: String,  
    val idade: Int,  
    val email: String,  
    val telefone: String,  
    var id: Int = 0          // var porque o BANCO gera o ID depois  
)
```

São **três níveis** de herança. `Instalador` herda de `Funcionario`, que herda de `Pessoa`. E `Cliente` e `Fornecedor` também herdam de `Pessoa`.

Diferença Kotlin × Java: em Java toda classe é aberta por padrão e você usa `final` para fechar. Em Kotlin é o contrário: tudo nasce fechado e você usa `open` para abrir. Isso evita herança acidental.

5.2 Polimorfismo

receberConta()

A mesma função, chamada do mesmo jeito, se comporta diferente conforme o tipo real do objeto:

```
// Pessoa.kt – comportamento padrão  
open fun receberConta(dinheiro: BigDecimal): BigDecimal = dinheiro  
  
// Cliente.kt – cliente paga, então para a EMPRESA o dinheiro ENTRA  
override fun receberConta(dinheiro: BigDecimal) = dinheiro.abs()      // +  
  
// Funcionario.kt – a empresa paga, então o dinheiro SAI  
override fun receberConta(dinheiro: BigDecimal) = dinheiro.negate()   // -  
  
// Fornecedor.kt – a empresa paga, dinheiro SAI  
override fun receberConta(dinheiro: BigDecimal) = dinheiro.negate()   // -
```

Onde isso aparece rodando: em `MenuFinanceiro.pagarSalario()`, a linha `funcionario.receberConta(funcionario.salario)` mostra na tela o efeito negativo no caixa.

5.3 Encapsulamento

o requisito mais cobrado

Encapsular = **esconder o dado e obrigar a passar por uma função que valida**. No trabalho isso aparece em três lugares:

a) O saldo do caixa

```
object Caixa {  
    // private = ninguém fora deste objeto enxerga.  
    // Se fosse público, qualquer linha poderia fazer  
    // Caixa.saldo = 1000000 e fraudar a empresa.  
    private var saldoInterno: BigDecimal = BigDecimal.ZERO  
  
    // A única forma de LER é esta função (BigDecimal é imutável)  
    fun saldo(): BigDecimal = saldoInterno  
}
```

b) O estoque do produto

```
class CaixaDaAgua(...) {  
    // "private set": todo mundo LÊ, só a classe ESCRIVE  
    var quantidadeEstoque: Int = quantidadeInicial  
    private set  
  
    // A única porta de saída – e ela valida  
    fun sairEstoque(quantidade: Int): Boolean {  
        if (quantidade <= 0) return false  
        if (quantidade > quantidadeEstoque) return false // não vende o que não tem  
        quantidadeEstoque -= quantidade  
        return true  
    }  
}
```

c) As parcelas do cliente

```
class Cliente(  
    // private val: a lista existe, mas ninguém de fora alcança  
    private val parcelasAPagar: MutableList<BigDecimal> = mutableListOf(), ...  
) {  
    // toList() devolve uma cópia IMUTÁVEL: dá para ler, não dá para alterar  
    fun listarParcelas(): List<BigDecimal> = parcelasAPagar.toList()  
  
    fun adicionarParcela(valor: BigDecimal) {  
        if (valor.signum() > 0) parcelasAPagar.add(valor) // valida antes  
    }  
}
```

5.4 Abstração

ConexaoPostgres

```
// "abstract" = classe BASE, não pode ser instanciada sozinha.
// Existe para ser herdada, e escreve a lógica de conexão UMA vez só.
abstract class ConexaoPostgres(
    val user: String = "postgres",
    val senha: String = "postgres",
    val url: String = "jdbc:postgresql://localhost:5432/caixadagua"
) {
    // protected = só esta classe e as filhas enxergam
    protected var c: Connection? = null

    fun conectar(): Boolean { ... }
    fun desconectar() { ... }
}
```

5.5 Interface genérica

InterfaceJPA<T>

```
// <T> é um TIPO GENÉRICO: um espaço em branco preenchido no uso.
// InterfaceJPA<CaixaDaAgua> → T = CaixaDaAgua
// InterfaceJPA<Movimentacao> → T = Movimentacao
interface InterfaceJPA<T> {
    fun salvar(item: T) // CREATE
    fun listar() // READ (mostra)
    fun listarTodos(): List<T> // READ (devolve)
    fun buscarPorId(id: Int): T? // READ (T? = pode não existir)
    fun editar(item: T, id: Int): Boolean // UPDATE
    fun excluir(id: Int): Boolean // DELETE
}

// Uma classe pode implementar UMA interface e herdar UMA classe:
class CRUDCaixaDAgua : InterfaceJPA<CaixaDaAgua>, ConexaoPostgres()
```

Interface × classe abstrata: a interface é um *contrato* — diz o QUE precisa existir, sem código. A classe abstrata é uma *base* — já traz código pronto para reaproveitar. Em Kotlin você herda **uma** classe, mas implementa **várias** interfaces.

5.6 Enum com construtor e função

```
enum class TipoMovimentacao(
    val descricao: String,
    val sinal: Int // +1 entra, -1 sai
) {
    VENDA_PRODUTO("Venda de produto", +1),
    COMPRA_PRODUTO("Compra de produto", -1),
    PAGAMENTO_SALARIO("Pagamento de salário", -1);

    fun ehEntrada(): Boolean = sinal > 0
}
```

A regra "isso entra ou sai?" mora **dentro do enum**. Nenhum menu decide sinal por conta própria. É por isso que o mesmo trecho de código serve para venda, compra e salário sem **if** espalhado.

5.7 Singleton com `object`

```
object Caixa { ... }    // existe UM caixa em todo o programa
object Sessao { ... }   // existe UMA sessão (quem está logado)
object Validador { ... } // existe UM validador
```

Não se escreve `Caixa()`, escreve-se `Caixa.saldo()`. É o equivalente ao `static` do Java. Faz sentido para o caixa porque a empresa tem **um** caixa só.

6. Segurança contra falha humana

São **quatro camadas** de proteção. Se perguntarem "como o sistema evita erro do usuário?", cite as quatro:

Camada 1 — REGEX (formato)

```
private val REGEX_CPF      = Regex("^\\d{11}$")
private val REGEX_CNPJ    = Regex("^\\d{14}$")
private val REGEX_EMAIL    = Regex("^\\[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\\.\\[A-Za-z]{2,}$")
private val REGEX_TELEFONE = Regex("^\\d{10,11}$")
private val REGEX_NOME     = Regex("^\\[A-Za-zÀ-ÿ ]{2,60}$")
```

Símbolo	Significado
<code>^</code>	começo do texto
<code>\$</code>	fim do texto
<code>\\d</code>	um dígito de 0 a 9
<code>\\D</code>	qualquer coisa que não é dígito
<code>{11}</code>	exatamente 11 vezes
<code>{10,11}</code>	de 10 a 11 vezes
<code>+</code>	uma ou mais vezes
<code>À-ÿ</code>	faixa de letras acentuadas

O `\\D` aparece numa jogada esperta: antes de validar, o sistema **limpa** a pontuação, então o usuário pode digitar de qualquer jeito:

```
// "123.456.789-09" vira "12345678909"
val entrada = lerLinha().replace(Regex("\\D"), "")
```

Camada 2 — NULLABLE (tipos seguros)

```
// toInt() QUEBRA o programa se o usuário digitar "abc".
// toIntOrNull() devolve null e o programa continua vivo.
val numero: Int? = lerLinha().toIntOrNull()
if (numero == null) { println("[ERRO] Digite um número."); continue }
```

Operador	Nome	O que faz
<code>?</code>	tipo nullable	<code>Int?</code> = pode ser número ou null.
<code>?.</code>	chamada segura	Só executa se não for null. <code>c?.close()</code>
<code>?:</code>	Elvis	Valor alternativo se der null. <code>x ?: "padrão"</code>
<code>!!</code>	not-null assertion	"Confie em mim, não é null." Quebra se for. Usado em <code>c!!.prepareStatement()</code> depois de <code>conectar()</code> ter dado <code>true</code> .
<code>as?</code>	cast seguro	Converte ou devolve null em vez de erro.

Camada 3 — TRY/CATCH (erros em execução)

```
val valor: BigDecimal = try {
    BigDecimal(texto)                // TENTA converter
} catch (e: NumberFormatException) { // se falhar, trata
    println("[ERRO] Valor inválido.")
    continue                        // volta e pergunta de novo
}
```

Nos CRUDs o padrão é `try / catch (SQLException) / finally`. O `finally` **sempre** roda, com erro ou sem erro — é lá que a conexão é fechada. E em `Main.kt` existe um `try/catch` em volta de tudo, como última rede.

Exceção	Quando acontece
<code>NumberFormatException</code>	Texto que não é número virando <code>BigDecimal</code> .
<code>SQLException</code>	Banco desligado, senha errada, CPF duplicado, chave estrangeira.
<code>ClassNotFoundException</code>	O <code>.jar</code> do driver não foi adicionado ao projeto.
<code>DateTimeParseException</code>	Data impossível, como <code>32/13/2025</code> .

Camada 4 — CHECK no banco

```
cpf CHAR(11) NOT NULL UNIQUE CHECK (cpf ~ '^[0-9]{11}$'),
setor VARCHAR(20) NOT NULL CHECK (setor IN ('FINANCEIRO', 'ADMINISTRATIVO',
                                             'LOGISTICA', 'INSTALACAO')),
quantidade_estoque INTEGER NOT NULL DEFAULT 0 CHECK (quantidade_estoque >= 0),
dimensao FLOAT8[] NOT NULL CHECK (array_length(dimensao, 1) = 3)
```

Por que validar duas vezes? Porque a validação do Kotlin protege contra o *usuário*, e a validação do banco protege contra *qualquer* programa — inclusive alguém rodando SQL direto no pgAdmin. O banco é a última linha de defesa.

7. O caminho do dinheiro (passo a passo)

Este é o percurso que mais cai em pergunta. Decore a sequência de uma **venda à vista**:

1. **Menu** — `venderCaixa()` confere se o operador pertence a um setor que aprova caixa.
2. **Filtro** — só aparecem produtos com `quantidadeEstoque > 0`.
3. **Validação** — a quantidade é lida com máximo igual ao estoque, então é impossível pedir mais do que existe.
4. **Cálculo** — `total = precoVenda × quantidade` em `BigDecimal`.
5. **Caixa** — `Caixa.receita(...)` recebe os 6 dados e faz 4 validações: valor positivo, pagador/recebedor preenchidos, responsável autorizado e saldo não negativo.
6. **Sinal** — o `Caixa` aplica `+` porque `VENDA_PRODUTO.sinal = +1`.
7. **Banco** — `salvarRetornandoId()` grava o INSERT e devolve o ID.
8. **Trava** — se o ID vier `null`, o saldo **não** muda e o estoque **não** baixa.
9. **Saldo** — só agora `saldoInterno = saldoFuturo`.
10. **Estoque** — `caixa.sairEstoque(qtd)` e depois `atualizarEstoque()` no banco.

Comparativo das operações

Operação	Caixa	Estoque	Observação
Comprar do fornecedor	– sai	↑ sobe	Se o caixa recusar, o estoque não sobe.
Vender à vista	+ entra	↓ desce	Dinheiro e mercadoria mudam juntos.
Vender parcelado	não mexe	↓ desce	Gera parcelas na dívida do cliente. Nada entra no caixa ainda.
Receber parcela	+ entra	não mexe	Se o caixa recusar, a parcela volta para a dívida.
Pagar salário	– sai	não mexe	Usa polimorfismo (<code>receberConta</code>).
Pagar despesa	– sai	não mexe	Luz, água, aluguel, boleto.
Agendar serviço	não mexe	não mexe	Só marca a data.
Concluir serviço	+ entra	não mexe	Cobra depois de executado — regra contábil correta.

8. Trechos que costumam ser cobrados

8.1 PreparedStatement e SQL Injection

```
// CERTO – os "?" são placeholders preenchidos pelo driver
val sql = "INSERT INTO movimentacao (valor, pagador, ...) VALUES (?, ?, ...)"
val stmt = c!!.prepareStatement(sql)
stmt.setBigDecimal(1, item.valor)
stmt.setString(2, item.pagador)

// ERRADO – juntar texto abre a porta para SQL Injection
val sql = "INSERT INTO movimentacao VALUES ('" + valor + "')"

```

Com `?`, o driver envia o dado **separado** do comando. Se o usuário digitar `' ; DROP TABLE cliente; --`, isso vira apenas um texto salvo, não um comando executado.

8.2 Enum ⇄ banco

```
// IR:      enum → texto
stmt.setString(4, item.cor.name)           // Cor.AZUL_FORTE → "AZUL_FORTE"

// VOLTAR: texto → enum
cor = Cor.valueOf(rs.getString("cor"))     // "AZUL_FORTE" → Cor.AZUL_FORTE
```

8.3 Array do PostgreSQL

```
// GRAVAR: MutableList<Double> → float8[]
val arrayDimensao = c!!.createArrayOf("float8", item.dimensao.toTypedArray())
stmt.setArray(3, arrayDimensao)

// LER: float8[] → MutableList<Double>
val dimensoes = (rs.getArray("dimensao")?.array as? Array<*>)
    ?.map { (it as Number).toDouble() }
    ?.toMutableList()
    ?: mutableListOf(0.0, 0.0, 0.0)      // Elvis: lista padrão se der null
```

8.4 O truque do `wasNull()`

```
val fornecedor = rs.getInt("fornecedor_id")
// getInt() devolve 0 tanto para o número zero quanto para NULL.
// wasNull() diz qual dos dois era.
val fornecedorId = if (rs.wasNull()) null else fornecedor
```

8.5 Pegar o ID gerado pelo banco

```
val stmt = c!!.prepareStatement(sql, Statement.RETURN_GENERATED_KEYS)
stmt.executeUpdate()
val chaves = stmt.generatedKeys
if (chaves.next()) item.id = chaves.getInt(1)
```

8.6 `executeUpdate` × `executeQuery`

Método	Usado em	Devolve
<code>executeUpdate()</code>	INSERT, UPDATE, DELETE	<code>Int</code> — quantas linhas foram afetadas. Se der 0 , o ID não existia.
<code>executeQuery()</code>	SELECT	<code>ResultSet</code> — a tabela em memória, percorrida com <code>while (rs.next())</code> .

8.7 Por que `COALESCE` no saldo

```
SELECT COALESCE(SUM(valor), 0) AS saldo FROM movimentacao
```

`SUM()` devolve `NULL` quando a tabela está vazia, e `NULL` quebraria a conta no Kotlin. `COALESCE` troca esse `NULL` por `0`.

9. Quarenta perguntas com resposta

Sobre o sistema e os requisitos

1. O que o sistema faz?

Gerencia uma empresa que vende e instala caixas d'água: controla o produto e seu estoque, o serviço de instalação, as pessoas envolvidas (funcionários por setor, clientes e fornecedores) e todo o fluxo de caixa, com tudo gravado no PostgreSQL.

2. Quantos setores existem e quais?

Quatro, no enum `Setor`: **Financeiro**, **Administrativo**, **Logística** e **Instalação**. O enunciado pedia no mínimo dois. Os dois primeiros têm `podeAprovar = true` e são os únicos que autorizam movimentação financeira.

3. Quais dados a movimentação financeira guarda?

Os seis exigidos: **valor** (quanto), **pagador** (quem pagou), **recebedor** (quem recebeu), **dataHora** (data e hora), **motivo** (a descrição) e **responsavel** (quem autorizou). Mais o `tipo` e o `id`.

4. De onde vem o "responsável"?

Do `object Sessao`. Ao iniciar, o programa pergunta qual funcionário está operando. `Sessao.nomeOperador()` devolve esse nome e ele é gravado em toda movimentação. Sem sessão, o sistema não saberia quem autorizou.

5. Onde está o encapsulamento do fluxo de caixa?

No `object Caixa`. O saldo é `private var saldoInterno`, ninguém de fora escreve nele. A única forma de mexer é chamar `registrar()` / `receita()` / `despesa()`, que validam antes, gravam no banco e só depois atualizam o saldo.

6. Por que o Caixa é object e não class?

Porque a empresa tem **um** caixa só. `object` cria um singleton: existe uma única instância no programa inteiro. Se fosse `class`, cada menu poderia criar seu próprio `Caixa()`, cada um com um saldo diferente — e o controle se perderia.

7. Quem decide se o dinheiro entra ou sai?

O enum `TipoMovimentacao`, pela propriedade `sinal` (+1 ou -1). Quem chama o caixa passa o valor **sempre positivo**; o `Caixa` consulta `tipo.ehEntrada()` e aplica `negate()` quando é saída. A regra mora num lugar só.

8. O que impede o caixa de ficar negativo?

Antes de gravar, o `Caixa` calcula `saldoFuturo = saldoInterno + valorComSinal`. Se `saldoFuturo.signum() < 0`, a operação é recusada e a função devolve `null`, sem tocar no banco nem no saldo.

9. O que acontece se o banco falhar no meio de uma venda?

`salvarRetornandoId()` devolve `null`. O `Caixa` detecta isso, **não** atualiza o saldo e devolve `null` para o menu. O menu, vendo `null`, **não** baixa o estoque. Resultado: ou tudo acontece, ou nada acontece.

10. Por que movimentação não pode ser editada nem excluída?

Por regra contábil: lançamento financeiro é histórico permanente. Os métodos `editar()` e `excluir()` do `CRUDMovimentacao` existem para cumprir o contrato da interface, mas imprimem "[BLOQUEADO]" e devolvem `false`. A correção se faz por **estorno** — um novo lançamento no sentido contrário.

11. Como funciona a venda parcelada?

O total é dividido pelo número de parcelas com `divide(n, 2, HALF_UP)`. Cada parcela é adicionada à lista privada do cliente com `adicionarParcela()` e gravada na coluna `parcelas_a_pagar`. **Nenhum** valor entra no caixa nesse momento — só quando cada parcela for recebida. Mas o estoque baixa, porque a mercadoria saiu.

12. Por que o serviço só gera receita ao ser concluído?

Porque o agendamento é só uma promessa. A receita é reconhecida quando o serviço é efetivamente executado — é o princípio contábil da competência. Enquanto está "AGENDADO", nada entra no caixa.

13. Por que a caixa d'água nasce com estoque zero?

Para que seja impossível ter produto que apareceu do nada. Unidade só entra no estoque através de uma **compra registrada** no financeiro, que gera a movimentação correspondente. Assim estoque e caixa sempre contam a mesma história.

Sobre Kotlin e POO

14. Diferença entre `val` e `var`?

`val` é somente leitura — atribuído uma vez e não muda mais (como `final` no Java). `var` pode mudar. No projeto, todos os campos de `Movimentacao` são `val` justamente porque lançamento financeiro é imutável. Já `id` é `var` porque o banco o gera depois.

15. Para que serve `open`?

Em Kotlin toda classe e todo método nascem **final**. `open` libera a herança (`open class Pessoa`) e a sobrescrita (`open fun receberConta`). Sem `open`, o `override` nem compila.

16. O que é override ?

A palavra que marca a substituição de um comportamento herdado. Em Kotlin ela é **obrigatória** (em Java, `@Override` é opcional), o que evita sobrescrever algo por engano.

17. Interface ou classe abstrata: quando usar cada uma?

Interface quando você quer só o **contrato** (o QUE) e a classe pode implementar várias. Classe abstrata quando você quer **reaproveitar código** (o COMO) e a classe só pode herdar uma. No projeto:

`InterfaceJPA` é o contrato CRUD; `ConexaoPostgres` é a base com o código de conexão pronto.

18. O que significa <T> em InterfaceJPA<T> ?

É um tipo genérico — um espaço em branco preenchido no momento do uso. Escrevemos o contrato uma vez e ele serve para todas as entidades: `InterfaceJPA<CaixaDaAgua>`, `InterfaceJPA<Movimentacao>`, etc. Sem genérico, seriam seis interfaces quase idênticas.

19. O que é companion object ?

O equivalente ao `static` do Java: o conteúdo pertence à **classe**, não ao objeto. Em `Movimentacao` ele guarda a constante `FORMATO` (o formatador de data/hora), que é a mesma para todas as movimentações e não precisa ser copiada em cada objeto.

20. O que é uma propriedade calculada?

Uma propriedade sem campo de memória, cujo valor é recalculado a cada leitura pelo `get()`. Exemplos: `Cliente.dividasAbertas` (true se a lista não está vazia), `Cliente.totalDevido` (soma das parcelas) e `CaixaDaAgua.lucroUnitario` (venda – custo). Nunca ficam desatualizadas.

21. Como funciona o when ?

É o `switch` do Kotlin, mais poderoso. Com argumento (`when (opcao) { 1 -> ... }`) compara valores. Sem argumento funciona como uma cadeia de `if/else if`: `when { qtd == 0 -> "SEM ESTOQUE"; qtd <= 3 -> "BAIXO"; else -> "" }`.

22. Diferença entre readln() e readlnOrNull() ?

`readln()` devolve `String` e lança exceção se a entrada acabar. `readlnOrNull()` devolve `String?` e simplesmente dá `null`. O `Validador` usa a segunda com Elvis: `readlnOrNull()?.trim() ?: ""`.

23. Por que toIntOrNull() em vez de toInt() ?

`toInt()` lança `NumberFormatException` e derruba o programa se o usuário digitar uma letra. `toIntOrNull()` devolve `null` e permite tratar o erro com uma mensagem educada e repetir a pergunta. É o coração do requisito "à prova de falha humana".

24. Para que serve o `!!` e por que ele é seguro aqui?

`!!` afirma "isto não é null" e quebra se for. Ele aparece em `c!!.prepareStatement()`. É seguro porque toda operação começa com `if (!conectar()) return` — ou seja, o código só chega no `!!` quando a conexão realmente foi aberta.

25. O que é o operador Elvis `? : ?`

Devolve o lado esquerdo se ele não for null, senão o lado direito. Exemplo do projeto: `operador?.nome ?: "SISTEMA"` — o nome de quem está logado, ou o texto "SISTEMA" se ninguém estiver.

26. O que `as?` faz?

É o cast seguro: tenta converter e, se não der, devolve `null` em vez de lançar `ClassCastException`. Usado ao ler os arrays do banco: `rs.getArray("dimensao")?.array as? Array<*>`.

27. Qual a diferença entre `filter`, `map`, `forEach`, `groupBy` e `fold`?

filter mantém só o que passa no teste (produtos com estoque). **map** transforma cada item em outra coisa. **forEach** percorre executando algo, sem devolver nada. **groupBy** monta um mapa agrupando por uma chave (funcionários por setor). **fold** percorre acumulando um resultado (soma das parcelas partindo de zero).

28. Qual é o ponto de entrada do programa?

A função `main()` em `Main.kt`. Ela chama `menuInicial()` dentro de um `try/catch/finally` global, que é a última rede de proteção: se algum erro escapar de todas as validações, o programa termina com mensagem clara em vez de stack trace.

Sobre o banco de dados

29. Como o Kotlin conversa com o PostgreSQL?

Via **JDBC**. O `Class.forName("org.postgresql.Driver")` carrega o driver e o `DriverManager.getConnection(url, user, senha)` abre a conexão. O driver é o arquivo `postgresql-42.x.jar` adicionado às bibliotecas do projeto.

30. O que significa cada parte da URL de conexão?

`jdbc:postgresql://localhost:5432/caixadagua` — **jdbc:postgresql** é o protocolo do driver, **localhost** é a máquina onde o banco está, **5432** é a porta padrão do PostgreSQL e **caixadagua** é o nome do banco.

31. O que é `PreparedStatement` e por que usá-lo?

É um comando SQL pré-compilado com placeholders `?`. Os valores são enviados separados do comando, o que **impede SQL Injection**, cuida da conversão de tipos e ainda é mais rápido quando o comando se repete.

32. O que é um `ResultSet` ?

O resultado de um `SELECT` — uma tabela em memória com um cursor apontando para uma linha por vez. `rs.next()` avança o cursor e devolve `false` quando acabam as linhas; por isso o padrão `while (rs.next())`.

33. O que é chave primária e chave estrangeira?

Chave primária (`SERIAL PRIMARY KEY`) identifica unicamente cada linha. **Chave estrangeira** (`REFERENCES`) liga uma tabela a outra e garante que o número apontado realmente existe. Na tabela `servico` há três: cliente, instalador e caixa.

34. Qual a diferença entre `ON DELETE RESTRICT` e `ON DELETE SET NULL` ?

RESTRICT recusa apagar o registro-pai enquanto houver filhos (não se apaga um cliente que tem serviço). **SET NULL** permite apagar e deixa a referência do filho como `NULL` (apagar um fornecedor não apaga as caixas dele, só as deixa sem fornecedor).

35. Por que a coluna `valor` não tem `CHECK (valor > 0)` ?

Porque o sinal já vem aplicado pelo Kotlin: entrada é positiva, saída é negativa. Um `CHECK > 0` impediria registrar qualquer despesa. O que existe é `CHECK (valor <> 0)`, que só barra lançamento de valor zero.

36. O que é a view `v_saldo_caixa` ?

Uma consulta salva que se comporta como tabela. Ela devolve o saldo atual, o total de entradas, o total de saídas e a quantidade de lançamentos, usando `SUM(...) FILTER (WHERE ...)`. Serve para conferir o saldo direto no pgAdmin, sem abrir o programa.

37. Como uma lista do Kotlin vira uma coluna do banco?

Pelo tipo array do PostgreSQL. `createArrayOf("float8", lista.toTypedArray())` transforma `MutableList<Double>` em `FLOAT8[]` (dimensões da caixa), e `createArrayOf("numeric", ...)` transforma `MutableList<BigDecimal>` em `NUMERIC(12,2)[]` (parcelas do cliente).

38. Por que `finally` nos CRUDs?

Porque `finally` executa sempre, com erro ou sem erro. É lá que fica `desconectar()`. Sem isso, uma exceção deixaria a conexão aberta, e conexões abertas acumuladas esgotam o servidor do banco.

39. O que `executeUpdate()` devolvendo 0 significa?

Que nenhuma linha foi afetada — normalmente porque o `id` informado no `WHERE` não existe. Por isso os CRUDs testam `if (linhas > 0)` e avisam "[AVISO] Nenhum registro com o ID X" em vez de fingir que deu certo.

40. Qual o maior risco do sistema e como ele foi tratado?

O maior risco é o saldo em memória divergir do saldo no banco. Foi tratado de três formas: (a) `sincronizarComBanco()` lê o saldo real ao iniciar; (b) o saldo em memória só muda **depois** que o INSERT é confirmado; (c) o extrato sempre lê do banco, nunca de uma lista em memória.

10. Roteiro de demonstração

Se pedirem para mostrar o sistema rodando, siga esta ordem — ela conta a história inteira em poucos minutos:

1. **Abrir** — mostre o login e explique que o operador escolhido vira o "responsável" de toda movimentação.
2. **Entrar com o instalador (Carlos)** e tentar pagar uma despesa. O sistema **bloqueia**. → prova a regra de setores.
3. **Trocar para Ana (Financeiro)** pela opção 5 do menu principal.
4. **Pessoas** → **opção 3**: funcionários agrupados por setor. → prova o requisito de setores.
5. **Financeiro** → **Comprar do fornecedor**: 10 unidades. Mostre o saldo caindo e o estoque subindo juntos.
6. **Tentar comprar 1000 unidades**: o caixa recusa por saldo insuficiente e o estoque **não** muda. → prova o encapsulamento.
7. **Financeiro** → **Vender à vista**: mostre o saldo subindo e o estoque caindo.
8. **Vender parcelado em 3x**: mostre que o caixa **não** mexe e a dívida aparece em "Ver clientes devedores".
9. **Receber uma parcela**: aí sim o dinheiro entra.
10. **Pagar salário**: mostre a linha do polimorfismo na tela.
11. **Ver extrato**: cada linha traz os seis dados exigidos.
12. **Digitar letra onde se espera número**, e um CPF inválido: o sistema pede de novo. → prova REGEX/TRY/NULLABLE.

11. Colinha final

Se perguntarem...	Responda...
Encapsulamento	<code>object Caixa</code> com <code>private var saldoInterno</code> ; estoque com <code>private set</code> ; parcelas com <code>private val</code> .
Herança	<code>Pessoa</code> → <code>Funcionario</code> → <code>Instalador</code> , três níveis. <code>Cliente</code> e <code>Fornecedor</code> também herdam de <code>Pessoa</code> .
Polimorfismo	<code>receberConta()</code> : positivo no <code>Cliente</code> , negativo no <code>Funcionario</code> e no <code>Fornecedor</code> .
Abstração	<code>abstract class ConexaoPostgres</code> , herdada pelos 6 CRUDs.
Interface	<code>InterfaceJPA<T></code> , genérica, com os 6 métodos do CRUD.
REGEX	CPF, CNPJ, e-mail, telefone e nome no <code>Validador</code> ; mais os <code>CHECK</code> no <code>schema.sql</code> .
TRY	<code>try/catch/finally</code> nos CRUDs, <code>catch (NumberFormatException)</code> no dinheiro e <code>try</code> global no <code>main()</code> .
NULLABLE	<code>Connection?</code> , <code>Int?</code> , <code>T?</code> em <code>buscarPorId</code> , <code>Movimentacao?</code> no retorno do <code>Caixa</code> .
Dinheiro	<code>BigDecimal</code> ⇔ <code>NUMERIC(12,2)</code> . Nunca <code>Double</code> .
Data e hora	<code>LocalDateTime</code> ⇔ <code>TIMESTAMP</code> , via <code>Timestamp.valueOf()</code> .
Segurança do SQL	<code>PreparedStatement</code> com <code>?</code> — impede SQL Injection.
Setores	4 no enum <code>Setor</code> ; só Financeiro e Administrativo aprovam caixa.

A resposta que serve para quase tudo: "Essa regra não está espalhada pelo sistema — ela mora num lugar só. Se precisar mudar, muda ali e vale para o programa inteiro." Vale para o sinal (no enum), para o saldo (no `Caixa`), para a validação (no `Validador`) e para a conexão (no `ConexaoPostgres`).